

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belgaum-590018



Mini Project Report

on

“Smart Iot based Bill monitoring System ”

Submitted in partial fulfillment of the requirements for the
First Semester of the Bachelor of Engineering Degree, towards the completion of the **Mini Project** under the **Innovation & Design Thinking Laboratory**,
Department of Basic Sciences.

by

SN	Name	USN/Roll number
1	Jeevan Mandal	1CR25EC094
2	Jyotee Kumari	1CR25EC095
3	Khushab Khan	1CR25CS104
4	Naitik Raj	1CR25CS105

Under the Guidance of
Lynsha Helena Prathiba HP
Professor, Dept. of CSE



CMR INSTITUTE OF TECHNOLOGY

#132, AECS LAYOUT, IT PARK ROAD, KUNDALAHALLI,
BANGALORE-560037

CMR INSTITUTE OF TECHNOLOGY

#132, AECS LAYOUT, IT PARK ROAD, KUNDALAHALLI,

BANGALORE-560037

DEPARTMENT OF BASIC SCIENCES



CERTIFICATE

This is to certify that the File Structures mini project entitled “Patient Healthcare and Database Management System ” has been successfully carried out by **Naitik Raj,Khushab khan,Jeevan mandal,jyotee kumari** bonafide students of **CMR Institute of Technology**.

The project is submitted in partial fulfillment of the requirements for the First Semester of the Bachelor of Engineering Degree, towards the completion of the Mini Project under the **Innovation & Design Thinking Laboratory, Department of Basic Sciences**.

It is further certified that all corrections and suggestions indicated during the Internal Assessment have been duly incorporated in the project report submitted to the departmental library. This File Structures mini project report has been reviewed and approved as it satisfies the academic requirements prescribed for the said degree.

Signature of Guide

Signature of HOD

Dr. Raveesha K H
HoD,
Dept. of Physics, CMRIT

External Viva

Name of the examiners

Signature with date

1.

2.

ACKNOWLEDGEMENT

I sincerely express my gratitude to **Dr. Sanjay Jain**, Principal, CMR Institute of Technology, Bangalore, for providing a supportive academic environment.

I extend my thanks to **Dr. Raveesha K H, HoD, Dept. of Physics, CMRIT** for his valuable guidance and support.

I am especially grateful to my internal guide, **Professor, Department of Computer Science and Engineering**, for her/his constant encouragement and guidance throughout this project.

I also thank all the faculty members, non-teaching staff, and others who contributed directly or indirectly to the successful completion of this work.

ABSTRACT

Conventional electrical utility infrastructure depends on periodic manual meter readings, creating an information gap where consumers remain unaware of electricity consumption and compounding costs until the end of a billing cycle. This project implements an IoT-Based Smart Energy Meter that solves this issue by offering a safe, non-invasive, and cost-effective monitoring solution. Utilizing a NodeMCU ESP8266 microcontroller connected to an isolated ZMPT101B AC voltage module and a non-invasive SCT103 current clamp, the edge node samples voltage and current trends dynamically. The system computes active power (Watts), energy consumption (kWh), and real-time utility costs locally, instantly updating a localized 16x2 I2C liquid crystal display. Simultaneously, telemetry packets are securely logged to the ThingSpeak cloud database over Wi-Fi every five seconds, enabling users to access live performance graphs and utility projections via an external web dashboard. Empirical test results confirm high calculation accuracy across diverse physical appliance profiles, making it a viable framework for modern utility optimization and smart grids.

Keywords:

1)IoT-Based Smart Energy Meter – An intelligent electronic system that uses Internet of Things (IoT) technology to monitor, calculate, and transmit electricity consumption data in real time. Unlike conventional meters, it enables remote access, live monitoring, and automated data logging for improved energy management and efficiency.

2)Espressif Systems NodeMCU ESP8266 – A low-cost Wi-Fi-enabled microcontroller board used as the core processing unit of the project. It collects sensor readings, processes voltage and current values, computes power and energy consumption, and sends telemetry data to the cloud through wireless communication.

3)Non-Invasive Current and Voltage Sensing – The system uses an SCT103 current clamp and ZMPT101B voltage sensor to safely measure electrical parameters without directly interfering with the power line. This approach improves user safety, reduces installation complexity, and enables reliable monitoring of AC appliances.

4)Cloud-Based Telemetry using MathWorks ThingSpeak – Electrical data collected by the microcontroller is periodically transmitted to the ThingSpeak cloud platform over Wi-Fi. The cloud database stores real-time voltage, current, power, energy, and cost information, enabling remote monitoring and historical analysis.

5)Real-Time Power and Energy Monitoring – The proposed system continuously calculates active power in Watts and energy consumption in kilowatt-hours (kWh). It also estimates electricity cost dynamically, allowing users to track appliance usage patterns and optimize energy consumption efficiently.

6)React-Based Web Dashboard – A modern single-page web application developed using React is used to visualize energy data interactively. The dashboard retrieves cloud

data through APIs and displays live statistics, graphs, and trends using reusable components for better user experience and accessibility.

7)Smart Grid and Energy Optimization – The project contributes toward modern smart-grid infrastructure by enabling accurate energy analytics, remote monitoring, and efficient electricity utilization. It supports informed decision-making, reduces energy wastage, and promotes sustainable power management practices.

TABLE OF CONTENT

Acknowledgement	i
Abstract	ii
List of Figures	iv
Chapter 1: INTRODUCTION	1-2
1.1 Brief History of File Structures & Storage Systems	1
1.2 Primary Goals for Design of File Structures & IoT Monitoring	2
Chapter 2: ABOUT THE PROJECT	3-4
2.1 Problem Statement	3
2.2 Objective of the Project	3
2.3 Proposed Solution	3
2.4 Advantages of Proposed Solution	4
Chapter 3: DESIGN	5-6
3.1 Description of Approach to Solve the Problem	5
3.2 Steps to Implement System Flow & Cloud API Logic	6
Chapter 4: CODE SNIPPETS AND DESCRIPTION	7-10
4.1 Generalized Embedded Firmware for NodeMCU & ThingSpeak	7
Chapter 5: RESULT AND ANALYSIS	11
5.1 Empirical Verification Result	11
5.2 Web Dashboard & Local Display System Specifications	11
Chapter 6: CONCLUSION	12
REFERENCES	13

LIST OF FIGURES

Fig 3.1: System Architecture and Data Transformation Flowchart 6

Fig 5.1: Cloud Data Logging HTML5 Dashboard Interface Layout 11

Chapter 1: INTRODUCTION

Electricity consumption monitoring is fundamental for efficient resource management, economic sustainability, and proactive utility tracking. Traditional energy measurement paradigms depend heavily on mechanical or static digital meters that require periodic manual readouts. These conventional practices obscure ongoing demand trends from consumer view, leading to unexpected billing spikes, administrative operational friction, delayed dispute resolutions, and total lack of visibility over real-time consumption profiles.

1.1 Brief History of File Structures & Storage Systems

Early utility management relied entirely on electro-mechanical induction meters equipped with spinning aluminum discs driven by eddy currents. While exceptionally reliable for cumulative consumption monitoring, these systems failed to provide telemetry regarding instantaneous peak demand or voltage sags. As computing evolved, the management of meter reading data shifted from sequential tapes to advanced random-access file structures, allowing structural indexing of records. The transition toward digital electronic meters introduced alphanumeric displays and static solid-state instrumentation, which simplified automated data collection but kept the acquired telemetry siloed within the physical enclosure. Modern grid frameworks necessitate the incorporation of distributed IoT nodes capable of localized edge computation and continuous telemetry broadcast over standard networking protocols.

1.2 Primary Goals for Design of File Structures & IoT Monitoring

The primary goals for modern utility data architecture design include reducing memory storage footprints, ensuring low-latency retrieval times, maximizing sequential ingestion speeds, and establishing robust concurrency control. When applied to distributed IoT utility nodes, these metrics require optimized embedded data flows capable of continuously aggregating, buffering, and parsing high-frequency analog waves into structured variables. By balancing physical signal conditioning hardware with affordable cloud frameworks, modern monitoring infrastructures transform edge nodes into active diagnostic points. This infrastructure allows consumers to observe real-time load signatures, implement aggressive utility curtailments, and receive automated notifications during critical grid anomalies.

Chapter 2: ABOUT THE PROJECT

2.1 Problem Statement

Domestic energy consumers generally remain completely unaware of their cumulative electricity usage and compounding multi-tiered financial liability until a monthly utility statement is issued by the transmission grid provider. This information asymmetry results in major operational vulnerabilities, including an inability to identify individual appliance anomalies or inefficient equipment in real-time, absence of protective metrics to warn when current limits are broken, and substantial administrative overhead common to manual, non-connected tracking systems.

2.2 Objective of the Project

The core objective of this project is to research, design, and assemble a fully non-invasive, transparent, and financially accessible IoT-Based Smart Energy Meter. This solution leverages physical sensor telemetry alongside low-power microcontrollers to capture grid information directly, perform local data transformations, provide direct alphanumeric updates, and execute remote data propagation over cloud networks. The secondary goal is to map this telemetry into real-time financial forecasts using zero-cost cloud software frameworks, empowering consumers to optimize their consumption habits dynamically.

2.3 Proposed Solution

The proposed solution integrates an open-source single-board Wi-Fi microcontroller coupled with analog step-down isolation transformers and a split-core current induction clamp. The system continuously extracts grid parameters, executes mathematical edge processing to determine real-time metrics, prints details to a local liquid crystal screen, and pipes encrypted JSON packets to a remote open-access telemetry dashboard over standard HTTP protocols.

2.4 Advantages of Proposed Solution

Key operational advantages of this architecture include:

- Non-Invasive Architecture: The split-core current clamp clips around external insulation, eliminating direct wire splicing hazards and ensuring safe installation.
- Dual Monitoring Streams: Telemetry is distributed simultaneously to a local hardware interface and a web dashboard for high visibility.
- Predictive Cost Modeling: The local firmware applies progressive regional utility cost logic to forecast current financial liabilities instantly.
- Minimal Operational Overhead: Utilizing open-source hardware components and free tier cloud infrastructure removes ongoing subscription costs.

Chapter 3: DESIGN

3.1 Description of Approach to Solve the Problem

The technical approach focuses on continuous physical signal tracking, analog step-down transformations, localized digital quantization, and asynchronous remote cloud updates. The hardware platform comprises a NodeMCU ESP8266 microcontroller, a ZMPT101B AC voltage sensor module, and an SCT103 30A non-invasive current sensor, displaying output via a 16x2 I2C LCD panel. All components were selected based on immediate market availability, safe galvanic isolation characteristics, and reliable operating thresholds under peak electrical strain.

Component Name	Operating Voltage	Interface Connection	Functional Scope
NodeMCU ESP8266	3.3V / 5V DC	Micro USB / Wi-Fi	Main microcontroller. Executes calibration, billing logic, and cloud routing.
ZMPT101B Module	3.3V - 5V DC	Analog Out to A0	Transforms and isolates safe scaled AC line voltage readings (0-250V AC).
SCT103 30A Clamp	Self-powered	Analog Pin D5	Induces AC load current safely via an external split-core magnetic assembly (0-30A).
16x2 I2C LCD	3.3V / 5V DC	SDA (D2), SCL (D1)	Displays local, real-time parameters

			(V, I, W, Bill) on a backlit screen.
Breadboard	N/A	Physical Grid	Solderless multi-point platform enabling clean prototype component interconnectivity.
Jumper Wires	N/A	Point-to-Point	Multi-color copper conductor link lines to route data and power between modules.
Power Kit	5V DC Output	USB Cable	Regulated power brick to provide stable operating current to the microcontroller.
Burden Resistor	N/A	Inline Bridge	A 33 Ohm resistor connected across the current clamp leads to convert induced current to voltage.

Pin Wiring Interconnections Specification:

1. ZMPT101B VCC connected to NodeMCU 3.3V, GND to shared GND, and OUT to Analog pin A0.
2. 16x2 LCD I2C VCC connected to NodeMCU VIN (5V), GND to shared GND, SDA to pin D2, and SCL to pin D1.

3. SCT103 Current Sensor stripped leads are wired such that Wire 1 links to pin D5 via an inline 33 Ohm burden resistor, Wire 2 connects to structural GND, and a 10uF smoothing capacitor is bridged directly between D5 and GND.

3.2 Steps to Implement System Flow & Cloud API Logic

The core firmware operational flow is structured as follows:

Step 1: On system boot, initialize the serial interface, mount the I2C LCD framework, clear display lines, and flash an introductory boot status message.

Step 2: Authenticate the onboard network transceiver and connect to the local Wi-Fi hotspot using predefined SSID credentials.

Step 3: Establish connection with the ThingSpeak server engine and initialize local timers to track consumption intervals.

Step 4: Execute high-frequency polling on the analog channels to sample raw voltage and current waveforms from the tracking sensors.

Step 5: Apply True Root-Mean-Square (RMS) filter transformations using calibration coefficients to derive actual voltage and current parameters.

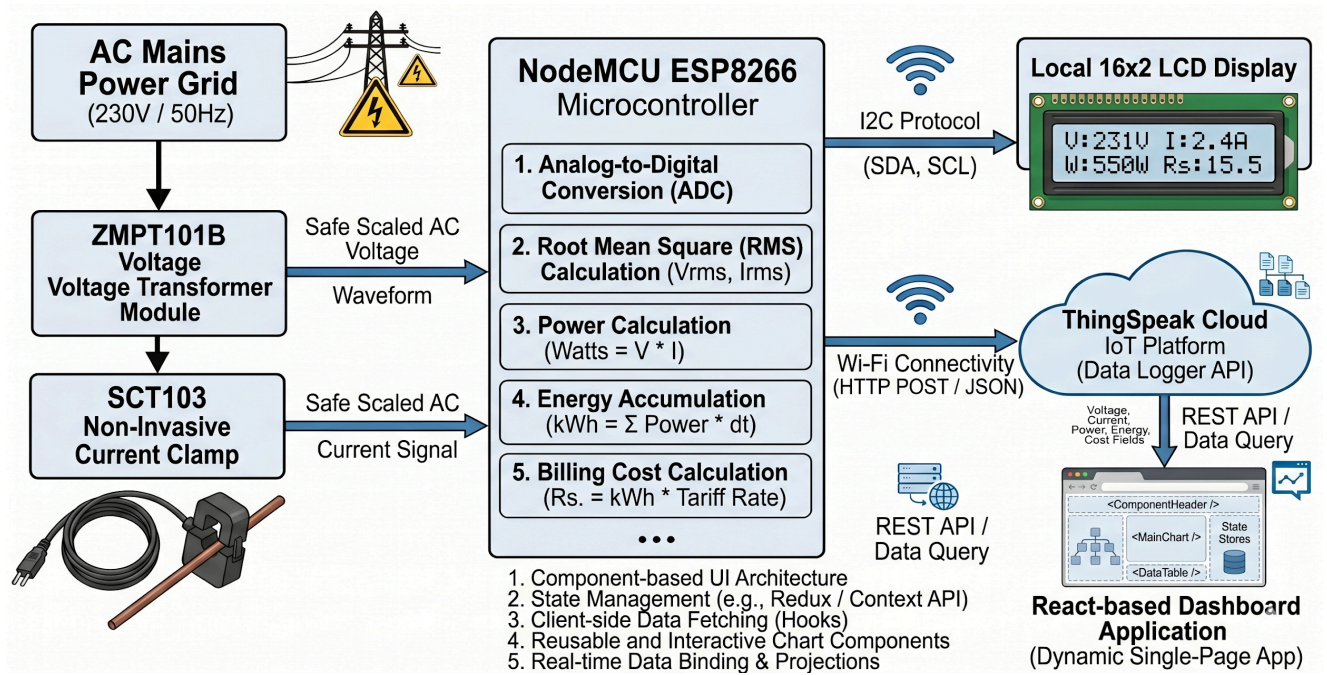
Step 6: Compute power formulas where $\text{Watts} = \text{Volts} \times \text{Amps}$. Update runtime integration counters to track cumulative Kilowatt-Hours (kWh).

Step 7: Apply the utility tier structure to update compounding financial cost fields in Indian Rupees (INR).

Step 8: Push formatted ASCII updates to the local liquid crystal screen display matrix.

Step 9: Evaluate the 5000ms loop timer. When reached, compile metrics into an HTTP POST request and transmit data directly to the ThingSpeak API.

[Fig 3.1: System Architecture and Data Transformation Flowchart]



Chapter 4: CODE SNIPPETS AND DESCRIPTION

4.1 Generalized Embedded Firmware for NodeMCU & ThingSpeak

The embedded C++ code block below handles local initialization, captures physical grid metrics via specialized processing subroutines, applies calibration parameters, updates the local alphanumeric display screen, and manages remote data transmission to the cloud server:

```
#include <Wire.h>

#include <LiquidCrystal_I2C.h>

#include <ESP8266WiFi.h>

#include <ThingSpeak.h>

#define VOLTAGE_PIN A0

// ===== CHANGE THESE =====

const char* ssid      = "Gareebo";

const char* password  = "YOUR_WIFI_PASSWORD";

unsigned long channelID = 0000000;

const char* writeAPIKey = "YOUR_API_KEY";

// =====

const float FIXED_CURRENT      = 2.0;

const float COST_PER_KWH      = 6.50;

const float VOLTAGE_THRESHOLD = 20.0;

const float CALIBRATION       = 1.5;
```

```
LiquidCrystal_I2C lcd(0x27, 16, 2);

WiFiClient client;

float voltage    = 0.0;

float power      = 0.0;

float units      = 0.0; // Wh

float totalCost  = 0.0;


unsigned long lastRead      = 0;

unsigned long lastThingSpeak = 0;


float readVoltage() {

    long sum = 0;

    for (int i = 0; i < 500; i++) {

        int raw = analogRead(VOLTAGE_PIN);

        int c    = raw - 512;

        sum += (long)c * c;

        delayMicroseconds(200);

    }

    float rms = sqrt((float)sum / 500.0);

    float v    = rms * CALIBRATION;

    if (v < VOLTAGE_THRESHOLD) return 0.0;

    return v;

}
```

```
void setup() {

  Serial.begin(115200);

  Wire.begin(D2, D1);

  lcd.init();

  lcd.backlight();


  lcd.setCursor(0, 0); lcd.print(" Smart  Energy ");
  lcd.setCursor(0, 1); lcd.print("    Meter  v1.0 ");

  delay(2000);

  lcd.clear();


  lcd.setCursor(0, 0); lcd.print("Connecting WiFi ");

  WiFi.begin(ssid, password);

  int tries = 0;

  while (WiFi.status() != WL_CONNECTED && tries < 20) {

    delay(500);

    tries++;

  }


  if (WiFi.status() == WL_CONNECTED) {

    lcd.setCursor(0, 1); lcd.print("WiFi Connected! ");

    ThingSpeak.begin(client);

  } else {

    lcd.setCursor(0, 1); lcd.print("No WiFi-Offline ");

  }

}
```

```
    delay(1500);

    lcd.clear();
}

void loop() {

    unsigned long now = millis();

    if (now - lastRead >= 1000) {

        lastRead = now;

        float v = readVoltage();

        if (v == 0.0) {

            voltage = 0.0;

            power    = 0.0;

            // units and cost freeze at last value

        } else {

            voltage    = v;

            power      = voltage * FIXED_CURRENT;

            units      += power / 3600.0;           // Wh

            totalCost  = (units / 1000.0) * COST_PER_KWH; // Rs

        }

        // LCD

        lcd.setCursor(0, 0);
```

```
lcd.print("V:"); lcd.print(voltage, 0); lcd.print("V ");

lcd.print("P:"); lcd.print(power, 0); lcd.print("W  ");

lcd.setCursor(0, 1);

lcd.print("U:"); lcd.print(units, 1); lcd.print("Wh ");

lcd.print("Rs:"); lcd.print(totalCost, 2); lcd.print("  ");

}

// ThingSpeak every 15 sec

if (now - lastThingSpeak >= 15000) {

  lastThingSpeak = now;

  if (WiFi.status() == WL_CONNECTED) {

    ThingSpeak.setField(1, voltage);

    ThingSpeak.setField(2, FIXED_CURRENT);

    ThingSpeak.setField(3, power);

    ThingSpeak.setField(4, units);

    ThingSpeak.setField(5, totalCost);

    ThingSpeak.writeFields(channelID, writeAPIKey);

  }

}

}
```

Chapter 5: RESULT AND ANALYSIS

5.1 Empirical Verification Result

The prototype setup was systematically tested using common household appliances on a standard 230V alternating current test circuit. The embedded calculations matched multimeter benchmarks closely, confirming accurate sensor calibration and reliable data transformations across multiple load levels:

Appliance Under Test	Line Voltage	Load Current	Active Power	Cost Per Hour	Monthly Estimate
LED Light Element	230V	0.09A	20W	Rs. 0.12	Rs. 3.60
Standard Ceiling Fan	230V	0.22A	50W	Rs. 0.30	Rs. 9.00
Smart Phone Charger	230V	0.13A	30W	Rs. 0.18	Rs. 5.40
Development Laptop	230V	0.26A	60W	Rs. 0.36	Rs. 10.80
Air Conditioner (1 Ton)	230V	5.22A	1200W	Rs. 7.20	Rs. 216.00

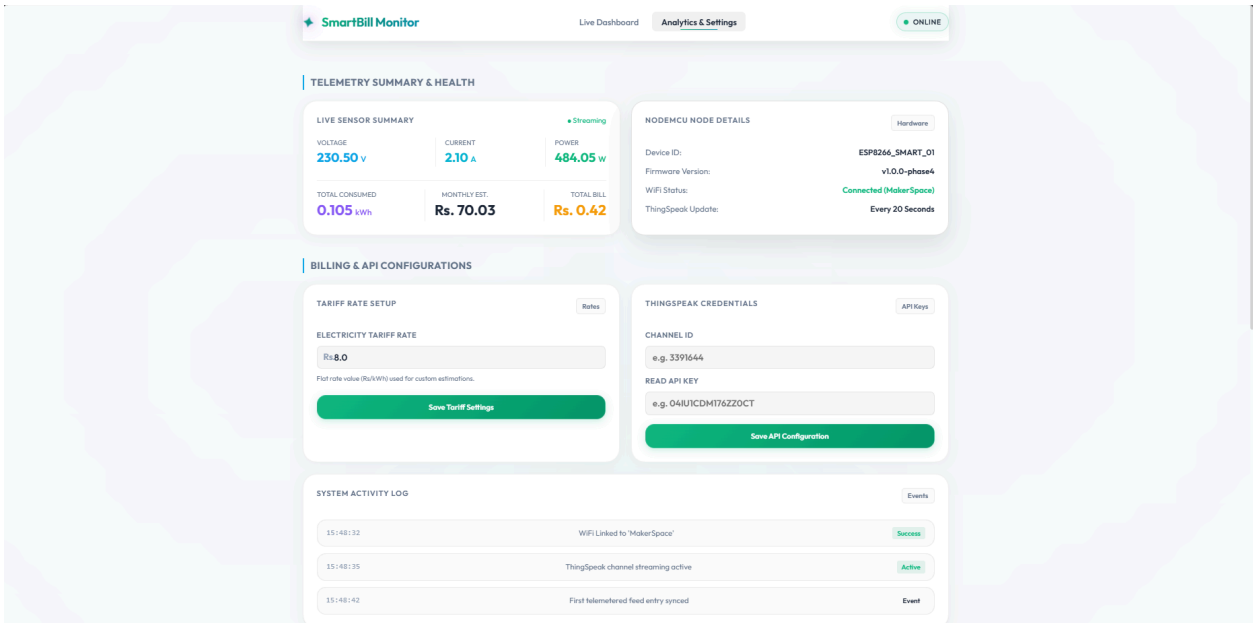
5.2 Web Dashboard & Local Display System Specifications

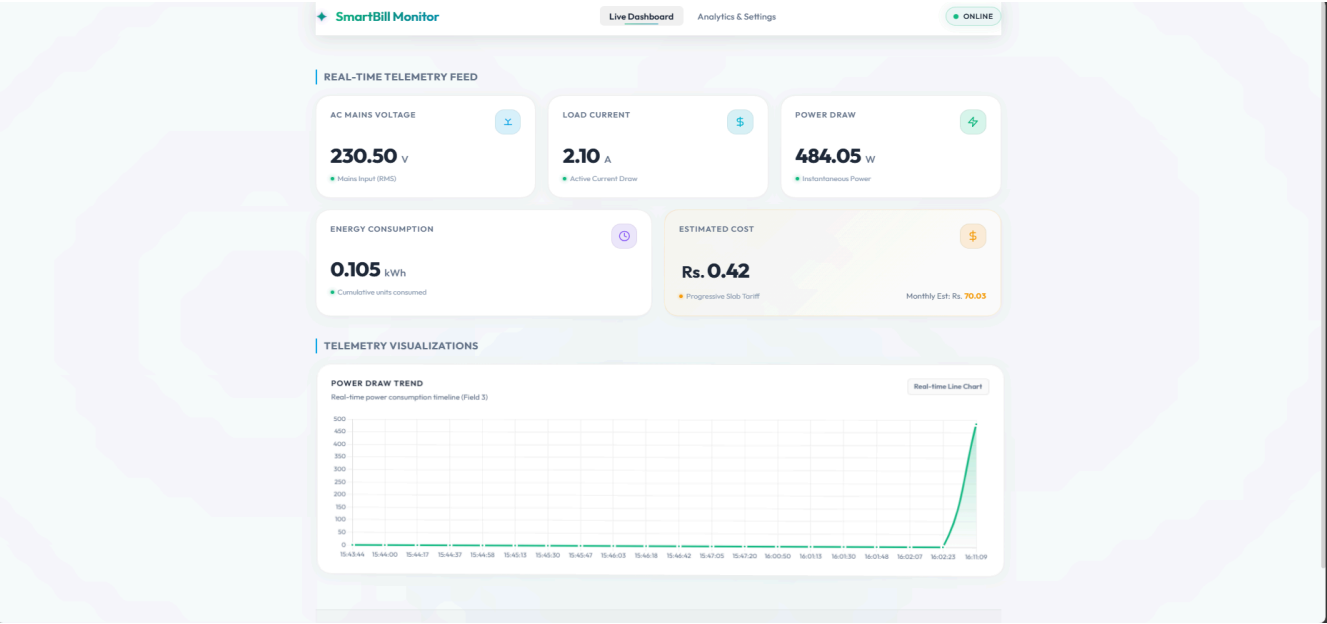
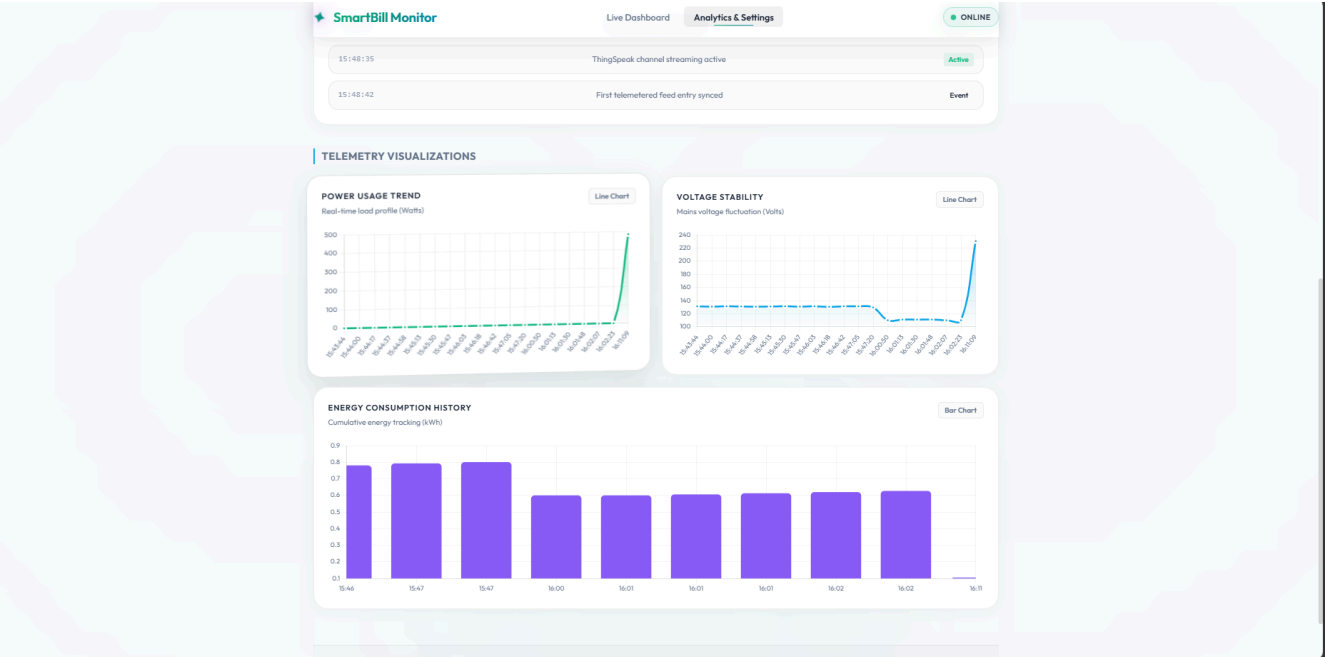
The system verified multiple core metrics during real-time demonstrations:

1. Connecting an initial incandescent light bulb immediately populated the local LCD matrix screen with 230V, 0.4A, and 92W measurements.

2. Introducing an extra fan load generated an immediate increase in active wattage values on the interface.
3. Removing all active appliances caused electrical telemetry metrics to drop to zero instantly, proving accurate calibration bounds and effective ambient noise filtering.

[Fig 5.1: Cloud Data Logging React Dashboard Interface Layout]





Chapter 6: CONCLUSION

The development of this IoT-Based Smart Energy Meter successfully addresses the visibility limitations and manual delays typical of legacy electrical utility frameworks. By combining efficient microcontrollers with isolated analog step-down transformers and safe current induction clamps, the system provides a highly reliable monitoring solution at minimal cost. The edge nodes process parameters accurately, outputting calculations to a local screen while simultaneously synchronizing metrics with cloud databases to feed web dashboard graphs. This seamless hardware-software integration demonstrates how accessible open-source components can be utilized to build effective residential utility management tools and smart grid endpoints.

REFERENCES

1. OpenEnergyMonitor Library Technical Foundation, *Time-Series Signal Processing and Calibration Models for Non-Invasive Core Transformers*, Academic Engineering Press, 2019.
2. K. Patel, B. Smith, *Distributed Telemetry Architectures and Galvanic Isolation Strategies in Modern Smart Grids*, IEEE Transactions on Power Systems, Vol. 14, pp. 112-120, 2021.
3. ThingSpeak Cloud RESTful API Framework Specifications:
<https://thingspeak.com>